Deformation-Aware Verlet Cache Specification

Stage S3 Variable-Cell Candidate Reuse for mdstats

mdstats

2026-07-14

Contents

1 Document contract

This document is the normative stage S3 specification for deformation-aware Verlet candidate reuse in `mdstats` version `0.14.0a3`.

It is written for two uses:

1. human review, maintenance, and scientific audit;
2. AI contextualization before later implementation work.

The Markdown and PDF versions contain the same normative content. Source code remains authoritative if a documentation defect is discovered.

Implementation files:

```
mdstats/analysis/_verlet_cache.py
mdstats/analysis/_cell_list.py
mdstats/analysis/_neighbors.py
```

Public imports:

```
from mdstats import (
    NeighborCacheStatistics,
    NeighborSearchSession,
    VerletCacheOptions,
    VerletPairCache,
)
```

2 Motivation

A conventional Verlet cache avoids rebuilding a neighbor candidate list at every frame. The cache is safe in a fixed cell while atomic motion has not consumed the skin distance. A changing simulation cell adds affine motion: compression, shear, and rotation can change pair separations even when fractional coordinates are unchanged.

Stage S3 extends the fixed-cell cache without weakening exactness. The design separates motion into:

- an affine cell map, bounded by its smallest singular value;
- nonaffine atomic motion in continuous fractional coordinates.

The resulting criterion is conservative, inexpensive, species-aware, and independent of the current candidate directions. It permits useful reuse under moderate cell deformation while rebuilding before an omitted pair can enter the physical cutoff.

3 Algorithmic provenance and originality boundary

The candidate buffer is the classical Verlet-list construction [1], and the fixed-cell displacement-triggered rebuilding inherited from S2 follows the automatic-update lineage represented by Chialvo and Debenedetti [2]. General parallelepiped neighbor lists [3] and cell-list algorithms for dynamically deforming periodic geometries [4] are related variable-cell prior art.

The complete S3 validity theorem is an mdstats derivation. In particular, the combination

```
smallest singular value of the affine cell map
+ species-resolved nonaffine endpoint displacement maxima
+ strict per-active-species-pair margins
```

is not attributed to any cited publication. References [3-4] establish the surrounding variable-cell neighbor-search literature, not the source of the formula implemented below.

4 Status and stage boundary

Stage S3 is implemented. Variable-cell reuse is explicit:

```
options = VerletCacheOptions(
    skin=0.5,
    deformation_aware=True,
)
session = NeighborSearchSession(collection, options)
```

The default remains the stage S2 fixed-cell policy:

```
VerletCacheOptions(deformation_aware=False)
```

With the default policy, any exact cell-matrix change rebuilds the cache.

Stage S3 changes only cache validity, stored rebuild references, diagnostics, and request identity. It does not change:

- the scientific neighbor definition;
- the strict cutoff rule `distance < cutoff`;
- minimum-image geometry;
- deterministic CSR ordering;
- the exact S1 cell-list rebuild backend;

- the blocked dense reference backend;
- hysteretic bond-state ownership.

The following remain outside S3:

- automatic backend selection inside this S3 module;
- consumer integration owned by this S3 module;
- a shared process-wide cache service;
- superset-cache reuse between different requests;
- changing atom count, atom identity, species, PBC flags, or selection scope;
- multi-image neighbor semantics;
- concurrent mutation of one session;
- inferred temporal continuity for independent ensembles;
- directional deformation bounds based on cached pair vectors.

5 Coordinate and data conventions

5.1 Row-vector cell convention

The three lattice vectors are rows of the cell matrix:

$$H_t = \begin{pmatrix} \mathbf{a}_t^{\mathsf{T}} \\ \mathbf{b}_t^{\mathsf{T}} \\ \mathbf{c}_t^{\mathsf{T}} \end{pmatrix}.$$

Fractional row vectors map to Cartesian coordinates by

$$\mathbf{r}_i(t) = \mathbf{s}_i(t) H_t.$$

For a trajectory, `AtomisticFrameCollection.fractional_positions` must be continuous through periodic boundary crossings. For an independent ensemble, fractional positions are wrapped separately in each frame and do not define a time-continuous branch.

5.2 Units

All distances, cells, positions, cutoffs, skins, and vectors are in angstrom. The implementation does not perform unit conversion inside the cache module.

5.3 Array ownership

Public result and cache arrays are defensive copies and read-only after construction. Callers must not depend on mutating returned arrays.

6 Public data structures

6.1 `VerletCacheOptions`

```python
@dataclass(frozen=True, slots=True)
class VerletCacheOptions:
    skin: float = 0.5
    safety_tolerance: float = 1.0e-12
    deformation_aware: bool = False
    max_cell_condition_number: float = 1.0e12
    cell_list_options: CellListOptions = CellListOptions()
```

Field	Type	Meaning	Constraint
`skin`	`float`	Cartesian margin added to the physical cutoff.	Finite and strictly positive.
`safety_tolerance`	`float`	Numerical reserve required in addition to the mathematical positive-margin condition.	Finite, nonnegative, and strictly smaller than `skin`.
`deformation_aware`	`bool`	Enables the S3 variable-cell validity test.	Must be boolean.
`max_cell_condition_number`	`float`	Largest accepted cell 2-norm condition number in S3 checks.	Finite and greater than `1`.
`cell_list_options`	`CellListOptions`	Exact S1 candidate-builder settings used on rebuild frames.	Must be a `CellListOptions` instance.

Methods:

`options.to_dict() -> dict[str, Any]`

`to_dict()` returns a JSON-oriented diagnostic representation. It is not a stable serialization format for reconstructing sessions across package versions.

6.2 `VerletPairCache`

```
@dataclass(frozen=True, slots=True)
class VerletPairCache:
    request_digest: str
    reference_frame_index: int
    selected_atom_indices: NDArray[np.int64]
    reference_wrapped_positions: NDArray[np.float64]
    reference_fractional_positions: NDArray[np.float64]
    reference_cell: NDArray[np.float64]
    active_pair_atomic_numbers: NDArray[np.int64]
    active_pair_cutoffs: NDArray[np.float64]
    center_indices: NDArray[np.int64]
    candidate_neighbor_indices: NDArray[np.int64]
    candidate_offsets: NDArray[np.int64]
    physical_cutoff: float
    list_cutoff: float
    pair_counting: PairCounting
    skin: float
    canonical_schema_version: str = "mdstats.verlet-cache.v2"
```

This object is created by `NeighborSearchSession`; users normally inspect it but do not instantiate it manually.

Field	Shape or form	Contract
`request_digest`	64-character lowercase hexadecimal string	SHA-256 digest of the exact normalized request.
`reference_frame_index`	scalar integer	Nonnegative frame at which candidates were last rebuilt.
`selected_atom_indices`	`(n_selected,)`	Nonempty, strictly increasing union of center and candidate selections.
`reference_wrapped_positions`	`(n_selected, 3)`	Finite Cartesian rebuild positions used by the fixed-cell fallback.

Field	Shape or form	Contract
`reference_fractional_positions`	`(n_selected, 3)`	Finite rebuild fractional coordinates; continuous for trajectory use.
`reference_cell`	`(3, 3)`	Finite row-vector rebuild cell.
`active_pair_atomic_numbers`	`(n_pair_types, 2)`	Positive, canonical, unique, lexicographically sorted atomic-number pairs.
`active_pair_cutoffs`	`(n_pair_types,)`	Positive finite physical thresholds aligned with active pair types.
`center_indices`	`(n_centers,)`	Normalized center atoms in request order.
`candidate_neighbor_indices`	`(n_cached_pairs,)`	CSR neighbor payload containing exact rebuild candidates.
`candidate_offsets`	`(n_centers + 1,)`	Nondecreasing CSR offsets, starting at zero and ending at `n_cached_pairs`.
`physical_cutoff`	scalar float	Positive finite scientific cutoff.
`list_cutoff`	scalar float	`physical_cutoff + skin`; strictly larger than `physical_cutoff`.
`pair_counting`	`PairCounting`	Directed or unordered-identical pair retention.
`skin`	scalar float	Positive cache skin.
`canonical_schema_version`	string	Must equal `mdstats.verlet-cache.v2`.

Properties and methods:

```
cache.n_candidate_pairs -> int
cache.summary() -> dict[str, Any]
```

`summary()` omits full coordinate and CSR arrays. It is intended for audit logs, not reconstruction.

6.3 NeighborCacheStatistics

```
@dataclass(frozen=True, slots=True)
class NeighborCacheStatistics:
    evaluations: int
    rebuilds: int
    reuse_evaluations: int
    candidate_pair_evaluations: int
    accepted_pairs: int
    current_candidate_pairs: int
    rebuild_reasons: tuple[tuple[str, int], ...]
```

All counters are nonnegative integers. Rebuild-reason entries are lexicographically sorted, have positive counts, and sum to `rebuilds`.

Required identities:

$$N_{\text{evaluations}} = N_{\text{rebuilds}} + N_{\text{reuse}}.$$

Properties and methods:

```
stats.mean_evaluations_per_rebuild -> float
stats.acceptance_ratio -> float
stats.to_dict() -> dict[str, Any]
```

The acceptance ratio is

$$\frac{N_{\text{accepted pairs}}}{N_{\text{candidate pair evaluations}}},$$

or zero when no candidate pair has been evaluated.

6.4 Returned `NeighborListResult`

`NeighborSearchSession.build_neighbor_list()` returns the shared immutable `NeighborListResult`:

```
@dataclass(frozen=True, slots=True)
class NeighborListResult:
    frame_index: int
    center_indices: NDArray[np.int64]
    neighbor_indices: NDArray[np.int64]
    offsets: NDArray[np.int64]
    vectors: NDArray[np.float64]
    distances: NDArray[np.float64]
    image_shifts: NDArray[np.int64]
    cutoff: float
    pair_counting: PairCounting
    backend: NeighborSearchBackend
```

The result is CSR-style. For center row `k`, neighbor entries occupy

```
start = result.offsets[k]
stop = result.offsets[k + 1]
```

with aligned `neighbor_indices`, `vectors`, `distances`, and `image_shifts`. Every returned distance satisfies the strict inequality

$$r_{ij} < r_{\text{cut}}.$$

The backend is `NeighborSearchBackend.VERLET_CACHE` even on a rebuild frame, because the user-facing operation is a cache-session evaluation.

6.5 Related enum values

The implementation uses the internal `PairCounting` enum with values:

```
directed
unordered_identical
```

The top-level package does not currently re-export `PairCounting`. User-facing calls should therefore pass the exact strings above unless internal APIs are being developed or tested. `unordered_identical` is valid only when the normalized center and candidate selections are identical. It retains one orientation using the atom-index ordering rule `center < neighbor`.

7 Public function and method calls

7.1 Construct a session

```
NeighborSearchSession(
    collection: AtomisticFrameCollection,
    options: VerletCacheOptions | None = None,
) -> NeighborSearchSession
```

Input contract:

- `collection` must be one validated, fixed-population `AtomisticFrameCollection`;
- every frame has the same atom count, atomic numbers, masses, and PBC flags;
- `cells` has shape `(n_frames, 3, 3)` and finite values;
- `fractional_positions` has shape `(n_frames, n_atoms, 3)` and finite values;
- trajectory fractional coordinates are time-unwrapped;
- `options=None` selects `VerletCacheOptions()`.

The session owns all request caches and statistics. It is intentionally mutable and not thread-safe.

7.2 Evaluate one frame

```
session.build_neighbor_list(
    *,
    frame_index: int,
    center_indices: ArrayLike,
    candidate_neighbor_indices: ArrayLike,
    cutoff: float | PairCutoff,
    pair_counting: PairCounting = PairCounting.DIRECTED,
) -> NeighborListResult
```

Argument	Accepted type	Constraint
`frame_index`	integer	Refers to exactly one valid frame; Python-style negative indexing is normalized by the shared validator.
`center_indices`	one-dimensional integer-like `ArrayLike`	Nonempty, unique, in bounds, and normalized by the shared selection validator.
`candidate_neighbor_indices`	one-dimensional integer-like `ArrayLike`	Nonempty, unique, in bounds, and normalized by the shared selection validator.
`cutoff`	positive finite `float` or `PairCutoff`	Must not exceed the conservative current-frame unique-image radius. `PairCutoff.radius` supplies the numeric value.
`pair_counting`	string `"directed"` or `"unordered_identical"`; internal `PairCounting` values are also accepted	`unordered_identical` requires identical center and candidate selections.

Output:

- exact current-frame neighbors under the strict physical cutoff;
- deterministic CSR ordering consistent with the shared neighbor subsystem;
- current minimum-image vectors, distances, and original-cell image shifts;
- no stale geometric values from the rebuild frame.

7.3 Inspect statistics

```
session.statistics(
    request_digest: str | None = None,
) -> NeighborCacheStatistics
```

With `None`, statistics are aggregated over all request digests. With a digest, statistics are returned for that exact request. An unknown digest returns a zero-valued snapshot.

7.4 Inspect a cache

```
session.cache_for_request(
    request_digest: str,
) -> VerletPairCache
```

The digest can be obtained from

```
session.request_digests -> tuple[str, ...]
```

An unknown digest raises `KeyError`.

7.5 Clear a session

```
session.clear() -> None
```

This discards both candidate caches and statistics. Subsequent evaluations begin with `initial_build`.

7.6 Session properties

```
session.n_caches -> int
session.request_digests -> tuple[str, ...]
```

Request digests are sorted for deterministic inspection.

8 Request identity

Each exact request is keyed by a SHA-256 digest with schema `mdstats.verlet-request.v2`. Identity includes:

- collection atom count;
- complete atomic-number sequence;
- PBC flags;
- normalized center indices;
- normalized candidate indices;
- pair-counting mode;
- physical cutoff;
- skin and safety tolerance;
- `deformation_aware`;
- `max_cell_condition_number`;
- S1 `CellListOptions`.

Changing any identity field creates a different cache. The implementation does not attempt superset reuse or tolerance-based request matching.

9 Candidate construction

For physical cutoff r_{AB} and global skin $r_s > 0$, the rebuild list radius is

$$R_{AB} = r_{AB} + r_s.$$

This enlarged-list construction follows the classical Verlet buffer [1].

At rebuild frame t_0, the exact S1 cell list stores every request-eligible pair satisfying

$$r_{ij}(t_0) < R_{AB}.$$

The scientific result accepts only

$$r_{ij}(t) < r_{AB}.$$

The list radius is checked against the conservative unique-image limit when the cache is built. On a reuse frame, only the physical cutoff is checked against the current cell because no new cell-list search is performed. Completeness is then guaranteed by the validity bound below.

10 Deformation-aware validity theory

10.1 Rebuild reference

Let t_0 be the latest successful rebuild frame. The cache stores:

- $H_0 = H_{t_0}$;
- wrapped Cartesian reference positions;
- continuous fractional reference positions $\mathbf{s}_i(t_0)$;
- exact candidate atom pairs at the enlarged list radius;
- active species-pair identities and thresholds.

All checks are relative to t_0, not to the immediately preceding evaluated frame. Requests may therefore skip frames without invalidating the logic.

10.2 Affine deformation map

Define

$$F_t = H_0^{-1} H_t.$$

For row vectors, a rebuild Cartesian vector transforms affinely as

$$\mathbf{x}_{\mathrm{aff}}(t) = \mathbf{x}_0 F_t.$$

Let $\sigma_{\min}(F_t)$ be the smallest singular value. Then

$$\|\mathbf{x}_0 F_t\| \geq \sigma_{\min}(F_t)\|\mathbf{x}_0\|.$$

This scalar lower bound covers isotropic strain, orthorhombic strain, shear, combined deformation, and rigid rotation.

For a rigid rotation Q,

$$H_t = H_0 Q, \qquad F_t = Q, \qquad \sigma_{\min}(Q) = 1.$$

A pure cell rotation therefore consumes no affine margin.

10.3 Nonaffine atomic motion

Write

$$\mathbf{r}_i(t) = \mathbf{r}_i(t_0) F_t + \mathbf{u}_i(t).$$

With continuous fractional coordinates,

$$\Delta\mathbf{s}_i(t) = \mathbf{s}_i(t) - \mathbf{s}_i(t_0),$$

and

$$\boxed{\mathbf{u}_i(t) = \Delta\mathbf{s}_i(t) H_t}.$$

This removes affine cell motion and retains atomic motion relative to the deforming fractional frame.

For species A in the exact request selection union,

$$u_A^{\max}(t) = \max_{i \in A} \|\mathbf{u}_i(t)\|.$$

10.4 Active species pairs

Let $\mathcal{C}$ be the set of species pairs that can occur in the request.

For different center and candidate selections, $\mathcal{C}$ is the canonicalized Cartesian product of species present in the two selections.

For identical selections, $\mathcal{C}$ contains unordered species pairs that can be formed by two distinct selected atoms. A same-species pair (A, A) is omitted when only one selected atom has species A.

This avoids an unnecessary doubled endpoint bound for a same-species pair that cannot exist.

10.5 Accepted pair margin

For every active pair type $(A, B) \in \mathcal{C}$, define

$$M_{AB}(t) = \sigma_{\min}(F_t)(r_{AB} + r_s) - r_{AB} - u_A^{\max}(t) - u_B^{\max}(t).$$

This species-resolved singular-value margin is the mdstats-specific S3 result; references [3-4] are related variable-cell methods but do not supply this criterion.

The cache is reusable only when

$$\boxed{M_{AB}(t) > \varepsilon \quad \text{for every } (A, B) \in \mathcal{C}},$$

where ε is `safety_tolerance`. Equality rebuilds.

10.6 Completeness argument

An omitted pair of type (A, B) satisfies at rebuild time

$$\|\mathbf{x}_0\| \geq r_{AB} + r_s.$$

After affine deformation,

$$\|\mathbf{x}_0 F_t\| \geq \sigma_{\min}(F_t)(r_{AB} + r_s).$$

Endpoint nonaffine motion can reduce separation by at most

$$\|\mathbf{u}_j - \mathbf{u}_i\| \leq \|\mathbf{u}_i\| + \|\mathbf{u}_j\| \leq u_A^{\max} + u_B^{\max}.$$

Therefore,

$$r_{ij}(t) \geq \sigma_{\min}(F_t)(r_{AB} + r_s) - u_A^{\max} - u_B^{\max}.$$

When $M_{AB} > 0$, the omitted pair remains strictly outside the physical cutoff. The implementation requires the stronger numerical condition $M_{AB} > \varepsilon$.

10.7 Fixed-cell reduction

When $H_t = H_0$,

$$\sigma_{\min} = 1,$$

so

$$M_{AB} = r_s - u_A^{\max} - u_B^{\max}.$$

For one species,

$$M_{AA} = r_s - 2u_A^{\max},$$

which reduces to the stage S2 rule

$$2d_{\max} < r_s - \varepsilon.$$

11 Ensemble policy

An independent ensemble has no continuous fractional branch across frames. Stage S3 therefore uses this policy:

1. If the current cell exactly equals the rebuild cell, apply the stage S2 reference-relative minimum-image displacement bound.
2. If the cell changed, rebuild with reason `fractional_unwrapping_unavailable`.

The implementation does not infer a deformation-relative atomic displacement from independently wrapped ensemble frames.

12 Algorithm

12.1 High-level pseudocode

```
function build_neighbor_list(request, frame):
    normalize frame and selections
    validate pair-counting relation
    validate physical cutoff in current cell
    digest = hash(exact request identity)
    cache = lookup(digest)

    if cache does not exist:
        reason = initial_build
    else if deformation_aware is false:
        if current cell differs exactly from reference cell:
            reason = cell_changed
        else if 2 * fixed_cell_displacement >= skin - tolerance:
            reason = displacement_limit
    else:
        validate reference and current cells

        if collection is a trajectory:
            F = solve(reference_cell, current_cell)
            sigma_min = smallest_singular_value(F)
            u_i = (s_i(frame) - s_i(reference)) @ current_cell
            compute species maxima u_A_max
```

```
            compute every active pair margin M_AB

            if affine-only margin is not positive:
                reason = cell_deformation_limit
            else if full margin is not positive:
                reason = nonaffine_displacement_limit
        else if current cell equals reference cell:
            apply fixed-cell displacement rule
        else:
            reason = fractional_unwrapping_unavailable

    if reason exists:
        build exact S1 candidates at cutoff + skin
        replace cache atomically
        record successful rebuild reason
    else:
        record reuse

    recompute current MIC geometry for every cached pair
    retain only distance < physical cutoff
    return deterministic CSR NeighborListResult
```

12.2 Rebuild operation

A rebuild:

1. validates the enlarged list cutoff in the rebuild cell;
2. calls the exact S1 triclinic cell-list backend;
3. stores candidate pair identity, not rebuild-frame vectors;
4. stores fresh Cartesian and fractional references;
5. derives active species pairs from the normalized request;
6. replaces the old immutable cache only after successful construction.

A failed rebuild does not increment successful evaluation or rebuild counters.

12.3 Reuse operation

A reuse operation never trusts stale distances or image shifts. For each cached pair, it recomputes in the current frame:

- Cartesian minimum-image vector;
- distance;
- original-basis integer image shift.

It then applies the strict physical cutoff.

13 Rebuild reasons

Reason	Meaning
`initial_build`	No cache exists for the request digest.
`cell_changed`	Fixed-cell S2 policy is active and the exact cell matrix changed.
`displacement_limit`	The fixed-cell reference displacement bound was reached.
`cell_deformation_limit`	Affine deformation alone consumed the available pair margin.

Reason	Meaning
`nonaffine_displacement_limit`	Affine margin remained positive, but endpoint nonaffine motion consumed the remainder.
`fractional_unwrapping_unavailable`	An independent ensemble changed cell, so a continuous fractional displacement is undefined.
`nonfinite_deformation_margin`	A finite conservative margin could not be established; the old cache is discarded before rebuilding.

`NeighborCacheStatistics.rebuild_reasons` records successful rebuild counts by reason.

14 Cell validity and numerical constraints

Deformation-aware reuse evaluates

$$F_t = H_0^{-1} H_t$$

and singular values of small matrices. Both reference and current cells must:

- have shape `(3, 3)`;
- contain only finite values;
- satisfy the shared nonsingularity checks;
- have finite singular values;
- satisfy

  $$\kappa_2(H) \le \kappa_{\max},$$

  where `max_cell_condition_number` defaults to 10^{12}.

Singular or excessively ill-conditioned cells raise `InvalidCellGeometryError`. The code does not silently reuse under an unreliable margin.

15 Exceptions

The module may expose or propagate:

Exception	Typical cause
`InvalidVerletCacheOptionsError`	Nonpositive skin, invalid tolerance, invalid condition-number limit, or malformed options.
`IncompatibleVerletCacheError`	Manually constructed or corrupted cache state violates schema or array invariants.
`InvalidCellGeometryError`	Singular, nonfinite, malformed, or excessively ill-conditioned cell.
`InvalidNeighborCutoffError`	Cutoff is nonpositive or nonfinite.
`UnsafeNeighborCutoffError`	Physical or rebuild list cutoff exceeds the conservative unique-image limit.
`CoincidentAtomsError`	Two distinct selected atoms occupy the same position within numerical tolerance.
`CellListComplexityError`	Exact S1 stencil construction exceeds configured hard limits.
`TypeError`, `ValueError`, `IndexError`	Malformed options, selections, frame indices, pair-counting relation, or array inputs.

Exceptions from the exact S1 backend are not converted into approximate fallback behavior.

16 Required scientific and software invariants

1. A cached result must never omit a pair returned by a fresh dense or fresh cell-list search.
2. Candidate identity is separate from the current minimum-image image.
3. Current vectors, distances, and image shifts are recomputed on every frame.
4. The physical rule remains `distance < cutoff`.
5. A margin equal to tolerance rebuilds.
6. A rigid cell rotation with unchanged fractional coordinates may reuse.
7. A changed-cell ensemble may not reuse through inferred unwrapping.
8. A singular or over-conditioned cell may not reuse.
9. Species maxima use the exact union of center and candidate selections.
10. A successful rebuild replaces all reference data consistently.
11. Request identity is exact; no approximate digest matching is permitted.
12. Hysteretic bond history remains outside the geometric cache.

17 Edge cases and warnings

17.1 Large compression or shear

A large deformation may invalidate the cache immediately, even when atoms have no nonaffine motion. This is expected. The bound is designed for safety, not to maximize cache lifetime under extreme deformation.

17.2 Large skins

A larger skin increases reuse time but also increases candidate count, memory, and per-frame candidate evaluation. It may also violate the unique-image limit at rebuild time. Skin selection is a performance parameter, not a scientific cutoff adjustment.

17.3 Very small skins

A skin close to the safety tolerance is invalid or causes nearly continuous rebuilds. Use a skin comfortably larger than floating-point noise and expected inter-rebuild motion.

17.4 Independent ensembles

Do not interpret ensemble frame order as a trajectory. Changed-cell ensemble frames rebuild by design. Convert data to trajectory semantics only when a physically continuous ordering and unwrapped fractional coordinates genuinely exist.

At the high-level S4 policy layer, `cache_mode="auto"` therefore keeps all independent ensembles stateless. A caller may explicitly request `cache_mode="verlet"` for a geometrically related fixed-cell ensemble; the exact S2 displacement bound still governs reuse. Repeated zero-reuse intervals are a performance pathology, not an accuracy error, so the policy disables that cache and returns to stateless cell-list execution after the configured limit.

17.5 Wrapped trajectory coordinates

The S3 nonaffine formula assumes time-unwrapped fractional coordinates. Supplying wrapped trajectory coordinates can create artificial jumps and excessive rebuilds. Worse preprocessing errors may also obscure the intended physical motion. Validate trajectory preprocessing before performance interpretation.

17.6 Atom or selection mutation

A session is bound to one fixed-population collection. Do not mutate atomic numbers, PBC flags, cells, or coordinate arrays in place after session creation. Create a new validated collection and session instead.

17.7 Request proliferation

Every exact selection, cutoff, pair-counting mode, or option set creates a separate request digest. Repeatedly creating nearly identical requests can grow memory usage. Reuse normalized requests deliberately and call `clear()` when a workflow is complete.

17.8 Threading and multiprocessing

One session is not thread-safe. Use one session per serial loop or one independent session per worker. Do not share mutable sessions across concurrent writers.

17.9 Pair-dependent cutoff expectations

The current call accepts one numeric radius per request. A `PairCutoff` carries provenance for one pair, but the cache does not yet accept a registry of different radii for multiple species pairs in one call. The internal active-pair arrays are structured for auditability and future extension; in S3 they align with the request's single physical radius.

17.10 Numerical conditioning

Raising `max_cell_condition_number` weakens a numerical guard; it does not improve the mathematical bound. Values near singular geometry should be fixed at the data or simulation level rather than admitted casually.

17.11 Performance interpretation

A high reuse count does not guarantee speedup. Candidate density, Python-level consumer overhead, atom count, and deformation path all matter. S3 establishes correctness. Automatic policy and cross-consumer benchmarking are implemented by S4 without changing this validity proof.

18 Complexity

Let N be the number of selected atoms, P_c the number of cached directed or unordered candidate entries, and S the number of active species pair types.

A reuse check costs approximately:

- $O(N)$ for nonaffine displacement norms and species maxima;
- $O(S)$ for pair margins;
- $O(P_c)$ for current minimum-image candidate evaluation.

The 3×3 solve and singular-value decomposition are constant-size operations.

A rebuild additionally pays the exact S1 cell-list construction cost. Memory is $O(N + P_c + S)$ per request cache.

19 Usage examples

19.1 Variable-cell trajectory

```
import numpy as np
from mdstats import NeighborSearchSession, VerletCacheOptions

selected = np.arange(collection.n_atoms, dtype=np.int64)

session = NeighborSearchSession(
    collection,
    VerletCacheOptions(
        skin=0.5,
        safety_tolerance=1.0e-12,
        deformation_aware=True,
        max_cell_condition_number=1.0e12,
```

```
    ),
)

for frame in range(collection.n_frames):
    neighbors = session.build_neighbor_list(
        frame_index=frame,
        center_indices=selected,
        candidate_neighbor_indices=selected,
        cutoff=3.2,
        pair_counting="unordered_identical",
    )
    # Consume neighbors without mutating its arrays.

print(session.statistics().to_dict())
```

19.2 Per-request inspection

```
digest = session.request_digests[0]
cache = session.cache_for_request(digest)
request_stats = session.statistics(digest)

print(cache.summary())
print(request_stats.to_dict())
```

19.3 Reset between independent workflows

```
session.clear()
assert session.n_caches == 0
```

20 Acceptance tests

The focused S3 test matrix compares

```
deformation-aware cached result
fresh exact S1 cell-list result
fresh blocked dense result
```

for:

- isotropic expansion;
- isotropic compression with positive margin;
- compression that crosses the affine margin;
- orthorhombic strain;
- volume-preserving shear;
- combined shear and nonaffine atomic motion;
- rigid cell rotation;
- nonaffine threshold crossing;
- species-aware framework/cation displacement maxima;
- periodic boundary crossing during deformation;
- an omitted pair placed arbitrarily close to the theoretical bound;
- randomized triclinic variable-cell trajectories;
- changed-cell independent-ensemble fallback;
- explicit ill-conditioned-cell rejection;
- all pre-existing fixed-cell and connectivity integration cases.

Release regression result:

```
272 passed, 24 expected warnings
```

21 AI implementation context

For later work, preserve these decisions unless a new specification explicitly replaces them:

```
- Cells and coordinates use row-vector convention: r = s @ H.
- Candidate rebuilds are exact S1 cell-list searches at cutoff + skin.
- S3 variable-cell reuse is opt-in, not automatic.
- F = solve(H0, Ht) = inv(H0) @ Ht.
- Nonaffine motion is (s_t - s_0) @ H_t using continuous trajectory fractions.
- Reuse requires every active species-pair margin to exceed tolerance strictly.
- Equality rebuilds.
- Rigid rotations should reuse.
- Changed-cell ensembles rebuild; do not infer unwrapping.
- High-level automatic cache reuse requires trajectory semantics.
- Explicit ensemble caching may be shut off after repeated zero-reuse intervals.
- Current MIC geometry is always recomputed.
- Session mutation is not thread-safe.
- S4 adds consumer integration and policy without weakening S3 exactness; later stages must preserve the sa
```

22 Completion criteria

Stage S3 is complete when:

- the margin in this document is implemented directly;
- every variable-cell path matches both fresh exact backends;
- rigid rotations avoid false rebuilds;
- affine and nonaffine rebuild causes are distinguishable;
- singular and over-conditioned cells are rejected explicitly;
- default fixed-cell behavior remains backward compatible;
- Markdown and PDF specifications are generated from the same source;
- source and installed-artifact checks remain successful.

Stage S4 is implemented and its semantics-aware policy is revised in `0.14.1` and preserves these invariants. Its public option, automatic policy, fallback, diagnostics, and consumer integration are specified in `docs/specs/analysis/neighbor_search_spec.md`.

23 References

1. Verlet, L. (1967). *Computer "Experiments" on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules*. Physical Review, 159(1), 98-103. DOI: 10.1103/PhysRev.159.98.
2. Chialvo, A. A., and Debenedetti, P. G. (1990). *On the Use of the Verlet Neighbor List in Molecular Dynamics*. Computer Physics Communications, 60(2), 215-224. DOI: 10.1016/0010-4655(90)90007-N.
3. Cui, Z., Sun, Y., and Qu, J. (2009). *The Neighbor List Algorithm for a Parallelepiped Box in Molecular Dynamics Simulations*. Chinese Science Bulletin, 54(9), 1463-1469. DOI: 10.1007/s11434-009-0197-0.
4. Dobson, M., Fox, I., and Saracino, A. (2016). *Cell List Algorithms for Nonequilibrium Molecular Dynamics*. Journal of Computational Physics, 315, 211-220. DOI: 10.1016/j.jcp.2016.03.056.